GORAD OS : Conception d'un Environnement d'exploitation Intelligent intégrant Cybersécurité, Intelligence Artificielle, Apprentissage et Développement personnel



Auteur *: RAKOTOARIVONY Anjarasoa Tsialavina Alifera*

Concepteur Système, Ingénieur Cybersécurité & CEO Founder @CyberGorad

Mail*: Rakotoarivonyalifera27@gmail.com*

Date *: [22/10/2026]*

Table des matières

Table des matières 2
Liste des Tableaux 4
Liste des Figures 5
Introduction 6
1. Présentation et analyse du problème 6
1.1 Contexte 6
1.2 Problème concret 6
1.3 Limites des systèmes existants 7
1.4 Objectifs du système 7
1.5 Périmètre du projet 8
2. Identification et formalisation des besoins 8
2.1 Acteurs du système 8
2.2 Exigences Fonctionnelles (EF) 9
2.3 Exigences Non-Fonctionnelles (ENF) 10
2.4 Contraintes 11
2.5 Besoins des utilisateurs (distincts des solutions techniques) 11
2.6 Exigences complémentaires 12
2.7 Critères de vérification et priorités 13
3. Décomposition fonctionnelle du système 14
3.1 Sous-système IDS GORAD (Sécurité) 15
3.2 Sous-système CYBERDRIVE (Formation et Aventure) 15
3.3 Sous-système AURAI (Développement Personnel) 16
3.4 Sous-système IA Locale (Moteur Cognitif) 17
3.5 Justification de la décomposition 17
3.6 Modules transversaux 18
3.7 Dépendances entre les modules 19
4. Modélisation des interfaces et des flux 20
4.1 Scénario d'interaction unifié (Exemple et illustration) 20

4.2 Points de contrôle et Risques d'erreur 20
4.3 Interactions entre acteurs et système (modèle de cas d'utilisation) 22
4.4 Communications entre composants et données échangées 24
4.5 Situations pouvant provoquer des erreurs, des incohérences ou des interruptions 25
5. Traçabilité des exigences 26
5.2 Matrice de traçabilité 27
6. Analyse des défaillances et validation 28
6.1 Analyse des modes de défaillance (AMDEC simplifiée) 28
6.2 Scénarios de test 29
6.3 Stratégie de validation 29
6.4 Défaillances complémentaires 30
6.5 Méthodes de vérification 32
6.6 Scénarios de test complémentaires 32
6.7 Critère de validation globale 34
7. Proposition d'architecture générale 35
7.1 Vue architecturale (en couches) 35
7.2 Technologies envisagées et justifications 35
7.3 Mesures de sécurité, fiabilité et maintenabilité 36
7.4 Composants logiciels et responsabilités 37
7.5 Interactions entre composants 38
7.6 Organisation des données et flux principaux 38
8. État d'avancement du prototype et perspectives 39
Conclusion 41
Références 41

Liste des Tableaux

Table 1 *: Besoins et responsabilités* 9
Table 2 *: Exigences Fonctionnelles* 10
Table 3 *: Exigences Non-Fonctionnelles* 10
Table 4 *: Besoins des utilisateurs* 12
Table 5 *: Exigences complémentaires* 12
Table 6 *: Critères de vérification et priorités* 14
Table 7 *: Modules transversaux* 19
Table 8 *: Dépendances modulaires* 20
Table 9 *: Analyse du modèle de cas d'utilisation* 22
Table 10 *: Communications entre composants* 25
Table 11 *: Cas d'erreurs* 26
Table 12 *: Traçabilité des exigences* 27
Table 13 *: Matrice de traçabilité* 28
Table 14 *: AMDEC* 29
Table 15 *: Défaillances complémentaire* 32
Table 16 *: Scénarios de test complémentaires* 34
Table 17 *: Architecture de GORAD OS* 36
Table 18 *: Composants logiciels* 37
Table 19 *: Organisation des données et Flux* 39

Liste des Figures

Figure 1 *: Tableau de bord de Surveillance de IDS-GORAD* 15

Figure 2 *: Tableau de bord de CYBERDRIVE* 16

Figure 3 *: AURAI dashboard* 17

Figure 4 *: Diagramme séquences FLUX* 21

Figure 5 *: Diagramme Cas utilisation* 23

Introduction

Aujourd'hui on va parler du projet GORAD OS ainsi que la conception détaillée de ce projet en Ingénierie des Systèmes et des Modèles. Il suit les sept étapes de l'énoncé : analyse du problème, formalisation des besoins, décomposition fonctionnelle, modélisation des interfaces et des flux, traçabilité, analyse des défaillances et validation, architecture générale. Les identifiants permettent de suivre un élément d'une section à l'autre : B (besoin), EF et ENF (exigences), M (module), R (risque), T (test).

Le projet repose sur un prototype existant (13 fonctionnalités recensées : 8 intégrées, 4 partiellement intégrées, 1 en développement ; 8 tests fonctionnels sur 10 réussis). Cet état d'avancement sert de point de départ et est détaillé en section 8.

1. Présentation et analyse du problème

1.1 Contexte

Les systèmes d'exploitation modernes (Windows, macOS, Linux) sont devenus des plateformes extrêmement riches, mais fragmentées. L'utilisateur moderne doit jongler entre plusieurs environnements : un outil pour la sécurité, des plateformes web pour la formation (cybersécurité), des applications tierces pour la productivité (gestion de tâches, calendrier), et des services d'Intelligence Artificielle (IA) généralement centralisés et dépendants du cloud (nécessitant une connexion Internet et soulevant des problèmes de confidentialité).

1.2 Problème concret

Cette fragmentation entraîne une déconnexion numérique : l'utilisateur disperse son attention, peine à maintenir des habitudes numériques saines (bien-être numérique), et n'a pas de vision unifiée de sa sécurité et de sa progression personnelle. De plus, les outils d'IA actuels collectent

massivement des données personnelles, ce qui pose un problème de souveraineté pour les données sensibles (comme les journaux de sécurité ou les notes personnelles).

1.3 Limites des systèmes existants

- **Sécurité :** Les systèmes d'exploitation grand public intègrent des antivirus, mais l'utilisateur ne comprend pas les alertes (effet "**boîte noire**").
- **Formation :** L'apprentissage de la cybersécurité se fait souvent sur des plateformes distantes, déconnectées de l'environnement réel de l'utilisateur.
- **IA :** Les LLM (Grands Modèles de Langage) sont majoritairement accessibles via des API distantes, ce qui n'est pas viable pour un environnement qui manipule des données critiques hors-ligne.
- **Développement Personnel (L'absence d'intégration native)** : a ce jour, il n'existe aucun système d'exploitation qui intègre nativement des outils avancés de développement personnel (gestion des habitudes, bien-être numérique, suivi cognitif). L'utilisateur est contraint d'installer des applications tierces fragmentées, qui ne communiquent pas avec son système.

1.4 Objectifs du système

Le projet **GORAD OS** vise à concevoir une distribution Linux spécialisée unifiant quatre piliers au sein d'un écosystème unique :

1. **Protection :** Surveillance et détection d'intrusion locale (IDS GORAD), cette projet est aussi conçu pour l’apprentissage du hacking éthique aussi.
2. **Formation :** Apprentissage gamifié de la cybersécurité (CYBERDRIVE) et aussi apprentissage du programmation un sous système en cours de

réalisation et conception, l'originalité est mis en priorité genre apprentissage avec des histoires.

3. **Développement personnel & Productivité :** Assistant de gestion de vie et de bien-être (AURAI).
4. **Assistance Souveraine :** IA locale fonctionnant hors-ligne pour garantir la confidentialité, utilisé aussi dans le système AURAI pour analyser et suivre la progression des utilisateurs.

1.5 Périmètre du projet

- **Inclus :** Conception de l'architecture modulaire, définition des flux de données entre les modules, spécification des interfaces utilisateur, intégration des mécanismes de sécurité.
- **Exclus :** Développement d'un nouveau noyau depuis zéro. L'entraînement *from scratch* [1]d'un modèle de langage (utilisation de modèles open-source existants). La commercialisation ou marketing après la réalisation d'un prototype fonctionelle.

2. Identification et formalisation des besoins

2.1 Acteurs du système

- **L'Utilisateur Final :** Personne cherchant à utiliser quelque chose de nouveau, à se former et à s'organiser pour avoir une vie plus alignée grace à l'informatique.
- **CyberGorad:** Fournit les mises à jour de GORAD OS et aussi les sous systèmes présent et les contenus pédagogiques (CYBERDRIVE).
- **Le Système (GORAD OS) :** Acteur chargé de la surveillance, la coordination des sous systèmes et aussi la communication entre elle et de l'exécution des tâches de fond (IA locale).

[1] Linux From Scratch (LFS) est une méthode qui consiste à compiler et assembler manuellement un système d'exploitation Linux complet à partir du code source, pièce par pièce, afin de comprendre son fonctionnement interne et d'obtenir un système sur mesure, optimisé et totalement maîtrisé.

Besoins et responsabilités de chaque acteur

Acteur	Responsabilités	Besoins principaux
Utilisateur final	Utilise l'environnement, suit les formations, valide ou refuse les actions proposées, règle les autorisations de données	B1 à B10
CyberGorad	Chargé pour la maintenance de toute les système pour le moment(recherche de collaboration)	B11, B12
Système (GORAD OS)	Collecte les événements, génère les alertes, exécute l'IA locale, journalise les actions automatiques	B1, B2, B7, B8

Table 1 : Besoins et responsabilités

2.2 Exigences Fonctionnelles (EF)

Réf.	Description	Priorité
EF01	Le système doit surveiller les processus, les ports et les connexions réseau en temps réel.	Critique
EF02	Le système doit générer une alerte visuelle lorsqu'un comportement suspect est détecté.	Critique
EF03	Le système doit proposer un parcours de formation structuré en modules avec laboratoire intégré, leçons et	Haute

Réf.	Description	Priorité
	quiz.	
EF04	Le système doit permettre de créer, suivre et organiser des tâches et des objectifs personnels.	Critique
EF05	Le système doit intégrer une IA locale capable de suivre la progression dans AURAI.	Haute
EF06	Les modules doivent communiquer entre eux.	Moyenne

Table 2 : Exigences Fonctionnelles

2.3 Exigences Non-Fonctionnelles (ENF)

Réf.	Description	Priorité
ENF01	Le système doit fonctionner entièrement hors-ligne (sauf pour les mises à jour).	Critique
ENF02	L'IA locale ne doit envoyer aucune donnée vers un service distant.	Critique
ENF03	Le système doit être basé sur un noyau Linux stable	Critique
ENF04	L'interface utilisateur est personnalisé par CyberGorad(style cyber, futuriste, 3D)	Haute
ENF05	Les composants doivent être modulaires (microservices ou modules applicatifs découplés) pour faciliter la maintenance.	Haute
ENF06	Le système doit résister aux pannes de courant ou aux bug imprévus sans corruption des données utilisateur.	Moyenne

Table 3 : Exigences Non-Fonctionnelles

2.4 Contraintes

- **Techniques :** Utilisation d'un noyau Linux existant, utilisation de LLM open-source (ex: Llama.cpp, Ollama).
- **Réglementaires :** Respect du RGPD par conception (Privacy by Design) grâce au traitement 100% local.
- **Économiques :** Utilisation exclusive de technologies open-source pour limiter les coûts.

2.5 Besoins des utilisateurs (distincts des solutions techniques)

Ces besoins expriment ce que veut l'utilisateur ; les solutions techniques (IDS, LLM local, seccomp, etc.) n'apparaissent que dans les exigences et l'architecture.

ID	Besoin exprimé
B1	Être averti quand quelque chose de suspect se passe sur ma machine
B2	Comprendre une alerte et savoir quoi faire
B3	Apprendre la cybersécurité progressivement, avec de la pratique et théorie
B4	Savoir où j'en suis dans ma formation
B5	Organiser mes objectifs, mes tâches, mon temps et mes notes
B6	Améliorer ma concentration et mes habitudes numériques sans être surveillé ni jugé
B7	Disposer d'une assistance intelligente sans envoyer mes données à l'extérieur
B8	Pouvoir travailler même sans connexion Internet

ID	Besoin exprimé
B9	Accéder à tout depuis un point d'entrée cohérent
B10	Garder la maîtrise de mes données et de toute action automatique
B11	Pouvoir administrer les comptes, les contenus et les réglages, mis à jours par le développeur
B12	Disposer d'un système stable, rapideet facile à utiliser

Table 4 : Besoins des utilisateurs

2.6 Exigences complémentaires

Réf.	Description	Priorité
EF07	Mis a jours du système et correction des bugs	Haute
EF08	L'utilisateur doit pouvoir visualiser les sources de données accessibles à l'IA dans AURAI.	Critique
EF09	Toute action automatique (ouverture d'une mission, création d'une tâche corrective) doit être confirmée par l'utilisateur et être annulable.	Haute
ENF07	Hors IA, l'ensemble IDS, CYBERDRIVE et AURAI doit rester léger (CPU et RAM au repos) et réactif (actions courantes en moins de 500 ms).	Haute
ENF08	Un nouvel utilisateur doit pouvoir réaliser les tâches de base sans aide (utilisabilité mesurée).	Moyenne

Table 5 : Exigences complémentaires

2.7 Critères de vérification et priorités

Les priorités sont celles des tableaux 2.2, 2.3 et 2.6. Les seuils chiffrés sont des cibles de conception à calibrer sur le matériel retenu.

Réf.	Critère vérifiable	Priorité
EF01	Sur 10 événements simulés (processus inconnu, port ouvert, connexion), au moins 9 sont enregistrés avec date, type, processus, adresse et priorité	Critique
EF02	Une alerte apparaît au tableau de bord ou en popup visible en 2 secondes maximum après le déclenchement, sur 20 alertes simulées	Critique
EF03	Les 8 modules et 54 leçons sont accessibles ; le score d'un quiz validé est enregistré	Haute
EF04	Création, modification et suppression d'une tâche et des objectifs ; données conservées après redémarrage	Haute
EF05	Emploi du temps optimisé géneré par l'IA d'après l'analyse comportementale de l'utilisateur et aussi resumé des alertes	Haute
EF06	Un clic sur une alerte ouvre la leçon associée ; la tâche corrective n'est créée qu'après confirmation	Moyenne
EF07	Affichage d'un tableau de bord centralisé qui affiche des donnés venant des sous systèmes	Haute
EF08	Graphes illustrant la progression de l'utilisateur	Critique
EF09	Aucune action automatique n'est exécutée sans confirmation ; chaque action figure dans un historique et peut être annulée	Haute

Réf.	Critère vérifiable	Priorité
ENF01	Réseau coupé, 100 % des fonctions locales testées fonctionnent (hors mises à jour)	Critique
ENF02	Capture réseau pendant 30 minutes d'utilisation de l'IA : aucune connexion sortante initiée par le service d'IA	Critique
ENF03	Noyau Linux Debian fonctionelle et stable	Critique
ENF04	Grub, Plymouth, Lanceur, thèmes complet dédié à GORAD OS	Haute
ENF05	Un module peut être remplacé en test sans modifier les autres ; interface de chaque module documentée	Haute
ENF06	Après coupure brutale pendant une écriture, aucune donnée validée n'est perdue ; un service arrêté redémarre en 30 secondes maximum	Moyenne
ENF07	Hors IA, moyenne d'au plus 10 % de CPU et 1 Go de RAM au repos pendant 1 heure ; actions courantes en moins de 500 ms	Haute
ENF08	Utilisateur de test satisfait et bon retour de sa part	Moyenne

Table 6 : Critères de vérification et priorités

3. Décomposition fonctionnelle du système

Le système est décomposé en 4 sous-systèmes majeurs, orchestrés par un noyau Linux personnalisé.

3.1 Sous-système IDS GORAD (Sécurité)

- **Rôle :** Collecter les journaux système, analyser les processus et le réseau, générer des alertes, Surveillance à distance d'un autre système.
- **Entrées :** Historique et log enregistré.
- **Sorties :** Événements de sécurité, alertes, logs formatés.
- **Dépendances :** Noyau Linux (Gorad Kernel).



Figure 1 : Tableau de bord de Surveillance de IDS-GORAD

3.2 Sous-système CYBERDRIVE (Formation et Aventure)

- **Rôle :** Fournir un environnement d'apprentissage gamifié (cours, quiz, missions).
- **Entrées :** Actions de l'utilisateur (clics, réponses), progression.
- **Sorties :** Contenu pédagogique, scores, badges.

- **Dépendances :** Réalisation du cours complet vérifié.

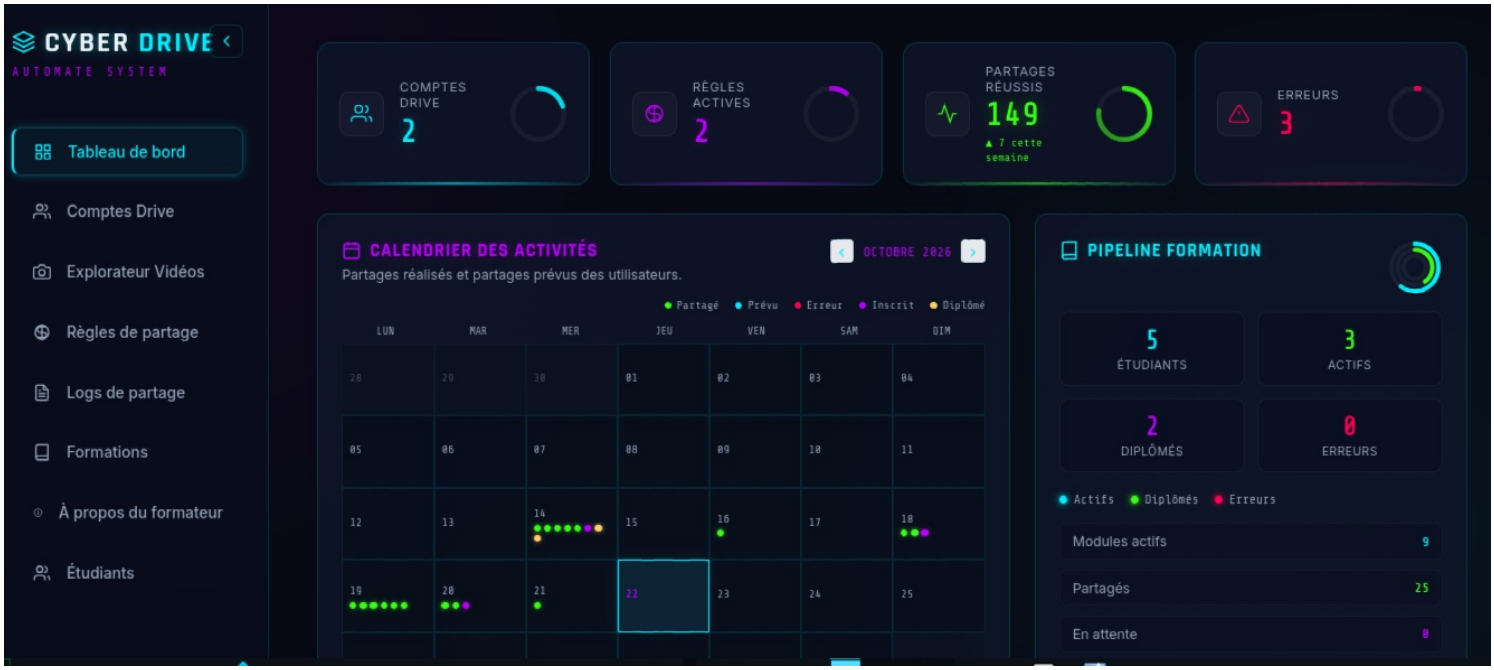


Figure 2 : Tableau de bord de CYBERDRIVE

3.3 Sous-système AURAI (Développement Personnel)

- **Rôle :** Gestion des tâches, des objectifs, du temps, et exercices de bien-être (concentration, Jeu pour entrainement cognitif, méditation, entrainement de prise parole). AURAI intègre aussi le «Computer Vision[2]» pour les entrainement comme Pomodoro ou méditation pour une suivi en temps réel des utilisateur et permettant de réaliser un système interactive entre l'utilisateur et AURAI.
- **Entrées :** Saisie utilisateur.

[2] «Computer Vision» est un domaine de l'IA qui permet aux machines d'analyser et de comprendre des images ou des vidéos, comme le ferait l'œil humain.

- **Sorties :** Tableau de bord de productivité, rappels, suivis de progression, optimisation par IA.
- **Dépendances :** Base de données locale, Saisie utilisateur.

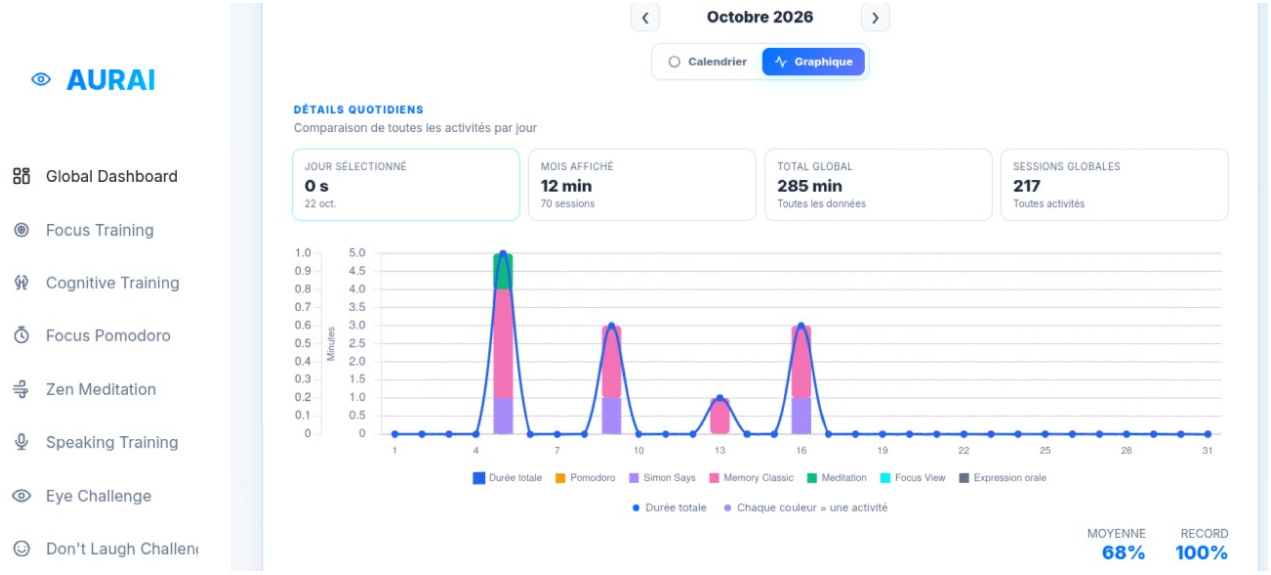


Figure 3 : AURAI dashboard

3.4 Sous-système IA Locale (Moteur Cognitif)

- **Rôle :** Exécuter un LLM local, traiter les requêtes en langage naturel, indexer et rechercher dans les données locaux (RAG).
- **Entrées :** Prompts utilisateur, documents locaux, alertes IDS.
- **Sorties :** Textes générés, résumés, explications.
- **Dépendances :** CPU/GPU, RAM (au moins 8 Go recommandés), bibliothèques d'inférence locale.

3.5 Justification de la décomposition

Cette approche par **séparation des préoccupations** (Separation of Concerns) permet de développer, tester et maintenir chaque module

indépendamment. L'interface entre ces modules se fait via une API interne (bus de messages ou API REST locale), garantissant qu'une défaillance du module CYBERDRIVE n'empêche pas IDS GORAD de protéger la machine.

3.6 Modules transversaux

En plus des quatre sous-systèmes métier, cinq modules communs rendent le système cohérent.

Module	Rôle et responsabilités	Entrées	Sorties
Noyau Linux et services système	Base du système : utilisateurs, services, démarrage	Configuration, demandes de services	Journaux, état des processus, ports, connexions
Tableau de bord et interface	Point d'accès unique, affichage des vues, recueil des confirmations	Données des modules via l'API, actions de l'utilisateur	Vues, confirmations
API interne	Faire communiquer les sous systèmes, valider les requêtes, journaliser les échanges	Requêtes	Réponses, événements transmis
Permissions, confidentialité	Authentification, rôles	Identifiants, choix de confidentialité	Décisions d'autorisation, journal d'accès
Stockage local	Conserver comptes, événements, progression,	Écritures des	Lectures,

Module	Rôle et responsabilités	Entrées	Sorties
(SQLite)	tâches, contenus	données	sauvegardes

Table 7 : Modules transversaux

3.7 Dépendances entre les modules

Module	Dépend de	Nature de la dépendance
IDS GORAD	Noyau Linux, API interne, Stockage	Sources d'événements, transmission et stockage des alertes
CYBERDRIVE	API interne, Stockage, Permissions	Contenus, scores, droits d'accès
AURAI	API interne, Stockage, Permissions ; CYBERDRIVE (facultatif)	Données de progression ; sans CYBERDRIVE, AURAI fonctionne mais n'affiche plus l'avancement de la formation
IA locale	API interne, Permissions, Stockage	Accès aux données uniquement après autorisation ; si l'IA est indisponible, les autres modules restent utilisables
Tableau de bord	API interne, Permissions	Accès à toutes les données via l'API
API interne	Permissions, Stockage	Contrôle d'accès, persistance
Permissions,	Noyau Linux	Services système et isolation

Module	Dépend de	Nature de la dépendance
Stockage		

Table 8 : Dépendances modulaires

4. Modélisation des interfaces et des flux

4.1 Scénario d'interaction unifié (Exemple et illustration)

1. **Détection :** IDS GORAD détecte une connexion réseau suspecte (ex: port 4444 ouvert).
2. **Alerte :** L'IDS envoie un événement au **Tableau de bord central**.
3. **Assistance (IA) :** L'utilisateur clique sur l'alerte. Le système invoque l'**IA Locale** pour expliquer la menace.
4. **Formation (CYBERDRIVE) :** L'IA suggère un lien vers le module CYBERDRIVE correspondant (ex: "Comprendre les chevaux de Troie").
5. **Action (AURAI) :** L'utilisateur termine la leçon. AURAI lui propose de planifier une tâche de "Mise à jour du pare-feu" dans son calendrier.

4.2 Points de contrôle et Risques d'erreur

- **Point de contrôle 1 (IDS -> UI) :** Risque de *spam* d'alertes. Prévoir un mécanisme de limitation de débit (rate limiting) dans l'interface.
- **Point de contrôle 2 (UI -> IA) :** Risque d'injection de prompt. L'IA locale doit être isolée (sandbox) pour ne pas exécuter de commandes système via l'interface de chat.
- **Point de contrôle 3 (AURAI -> Database) :** Risque de concurrence d'accès aux données. Nécessite des transactions ACID pour éviter la corruption.

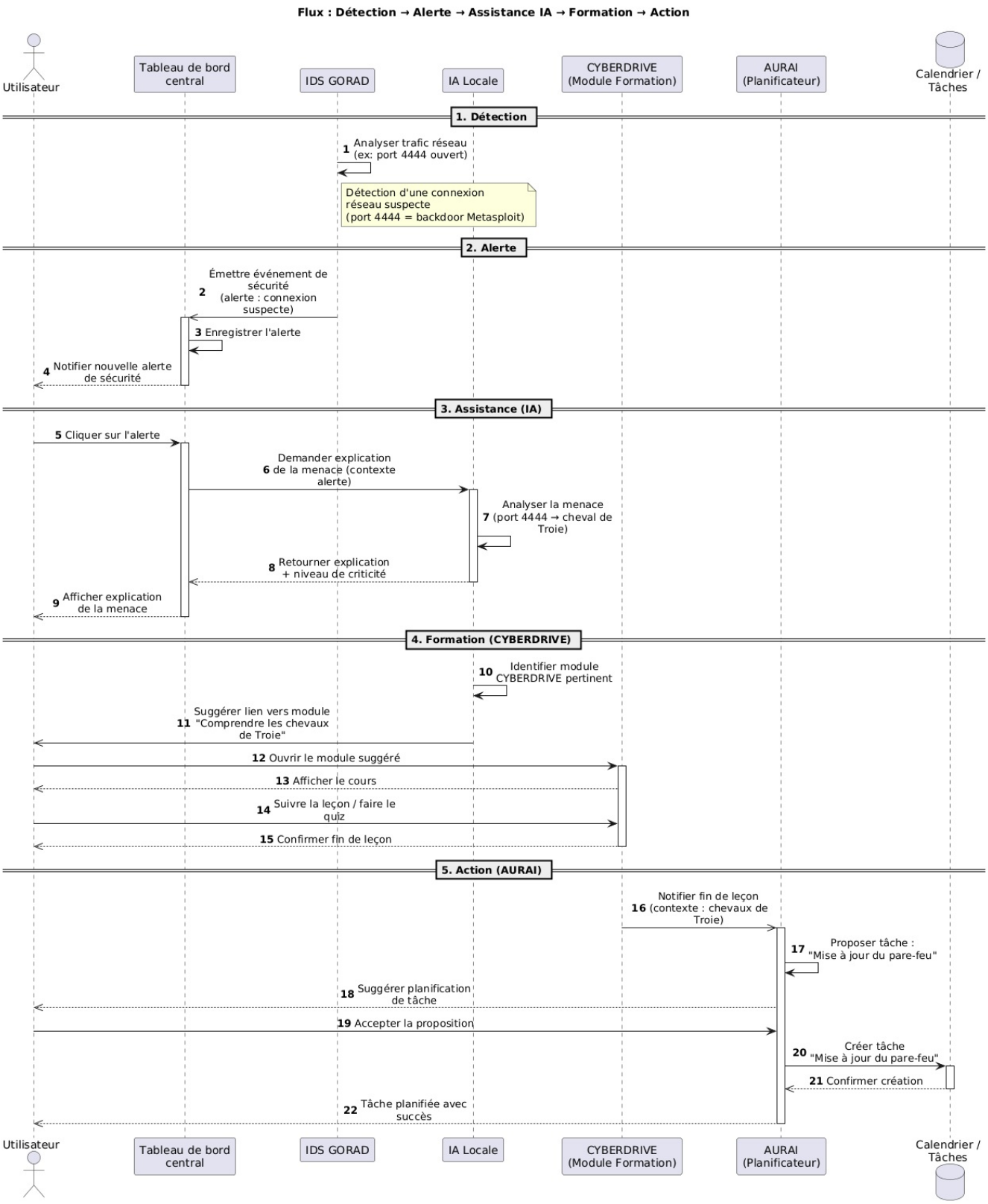


Figure 4 : Diagramme séquences FLUX

4.3 Interactions entre acteurs et système (modèle de cas d'utilisation)

Objectif du modèle : relier chaque acteur aux besoins et aux exigences. Il représente les acteurs (utilisateur, Cybergorad, Kernel GoradOS) et les cas d'utilisation qu'ils déclenchent.

Cas	Acteur	Cas d'utilisation	Besoin	Exigence
CU1	Utilisateur	Consulter les alertes de sécurité	B1	EF02
CU2	Utilisateur	Demander l'explication d'une alerte	B2	EF05
CU3	Utilisateur	Suivre un cours, faire un quiz, réaliser une mission	B3, B4	EF03
CU4	Utilisateur	Gérer objectifs, tâches, notes et exercices de bien-être	B5, B6	EF05
CU5	Utilisateur	Interroger l'IA locale (recherche, résumé)	B7, B8	EF05, ENF01
CU6	Utilisateur	interagis avec le système	B10	EF09
CU7	Utilisateur	Confirmer ou refuser une action proposée	B10	EF09
CU8	cybergorad	Développer système , corrige les Bug	B11	EF07
CU9	Système (GORAD OS)	Collecter journaux, processus, ports et connexions	B1	EF01

Table 9 : Analyse du modèle de cas d'utilisation

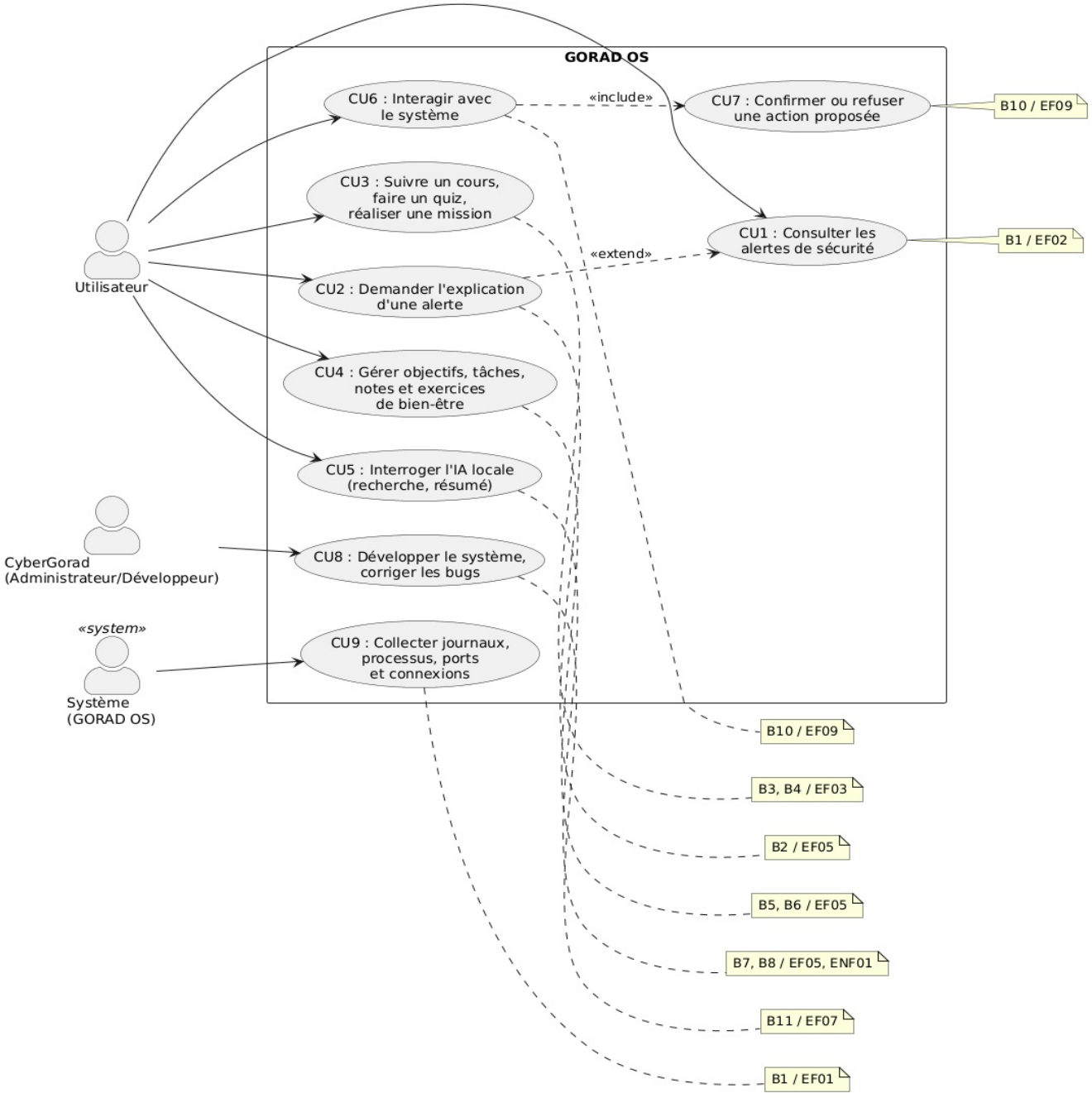


Figure 5 : Diagramme Cas utilisation

4.4 Communications entre composants et données échangées

Objectif : préciser qui échange quoi, d'où vers où, et où se situent les points de contrôle.

Flux	Origine	Destination	Données échangées	Point de contrôle
F1	Noyau Linux	IDS GORAD	Journaux, processus, ports, connexions	Lecture seule, filtrage des sources
F2	IDS GORAD	Stockage	Événement (date, type, machine, processus, adresse, priorité, état)	Validation du format
F3	IDS GORAD	Tableau de bord (via API)	Alerte à afficher	Limitation de débit (rate limiting)
F4	IDS GORAD	IA locale (via API)	Alerte à expliquer	Consentement vérifié par Permissions
F5	IA locale	Tableau de bord	Explication et recommandation, prédiction	Mention du caractère généré par IA
F6	Tableau de bord	CYBERDRIVE	Demande d'ouverture d'une leçon ou mission	Confirmation de l'utilisateur
F7	CYBERDRIVE	AURAI (via API)	Avancement, scores	Droit d'accès d'AURAI
F8	Tableau de bord	AURAI	Création d'une tâche corrective	Confirmation de l'utilisateur
F9	IA locale	Stockage	Lecture des	Sources autorisées uniquement, journal

Flux	Origine	Destination	Données échangées	Point de contrôle
			Donnéeslocaux	d'accès

Table 10 : Communications entre composants

4.5 Situations pouvant provoquer des erreurs, des incohérences ou des interruptions

Situation	Effet possible	Détection ou parade	Test
Source de journaux indisponible	L'IDS ne voit plus certains événements, Freeze de Tableau de bords	Contrôle de santé du collecteur, alerte de surveillance dégradée	T16
Événement mal formé	Alerte absente ou illisible	Validation du format avant stockage	T21
Rafale d'alertes	Interface saturée, alertes ignorées, spam alertes vocales	Limitation de débit et regroupement	T22
API interne indisponible	Modules isolés, interface vide	Mode dégradé avec message clair, reprise automatique	T16
Modèle de langage lent ou arrêt, Quota token atteint	Explication absente	Délai maximal, message de repli avec description technique de l'alerte	T20
Incohérence entre score CYBERDRIVE et progression	Avancement faux	Source unique : AURAI lit les données de CYBERDRIVE sans les dupliquer	T9

Situation	Effet possible	Détection ou parade	Test
AURAI			
Confirmation non reçue	Action en attente indéfinie	Expiration et annulation de la proposition	T8
Coupure d'alimentation pendant une écriture	Données corrompues	Écriture transactionnelle	T17

Table 11 : Cas d'erreurs

5. Traçabilité des exigences

Besoin métier	Exigence associée	Composant responsable	Méthode de vérification
Sécuriser l'utilisateur	EF01, ENF03	IDS GORAD, GORAD KERNEL	Tests d'intrusion (Pentest), vérification des logs
Former l'utilisateur	EF03	CYBERDRIVE	Tests fonctionnels (validation des quiz), retour utilisateur
Assister sans fuite de données	EF05, ENF02	IA Locale	Analyse du trafic réseau (IDS-GORAD) pendant l'utilisation de l'IA
Organiser la vie numérique	EF04	AURAI	Scénarios de test sur la persistance des tâches
Cohérence	EF06, ENF04	Tableau de	Tests d'intégration (bout en

Besoin métier	Exigence associée	Composant responsable	Méthode de vérification
globale		bord centralisé	bout)

Table 12 : Traçabilité des exigences

5.2 Matrice de traçabilité

Elle relie chaque besoin (section 2.5) aux exigences, aux composants et aux tests (section 6). Les tests T1 à T4 sont décrits en 6.2, les tests T5 à T22 en 6.6.

Besoin	Exigences	Composants	Tests
B1 Être averti	EF01, EF02, ENF03	IDS GORAD, Noyau Linux, Tableau de bord	T1, T5, T6
B2 Comprendre une alerte	EF05, EF06	IA locale, IDS GORAD, CYBERDRIVE	T7, T8
B3 Apprendre	EF03	CYBERDRIVE, Stockage	T4, T9
B4 Suivre sa progression	EF03, EF04	CYBERDRIVE, AURAI	T4, T9, T10
B5 S'organiser	EF04	AURAI, Stockage	T3, T10
B6 Concentration et bien-être sans contrôle	EF04, EF09	AURAI, Permissions	T10, T8
B7 Assistance IA sans fuite	EF05, EF08, ENF02	IA locale, Permissions	T11, T12
B8 Travail hors ligne	ENF01	Tous les modules	T2

Besoin	Exigences	Composants	Tests
B9 Point d'accès cohérent	EF06, ENF04, ENF08	Tableau de bord, API interne	T8, T18
B10 Maîtrise des données et des actions	EF08, EF09, ENF03	Permissions, IA locale, AURAI	T11, T14
B11 Administration	EF07	Permissions, Tableau de bord	T13
B12 Stabilité, légèreté, maintenance	ENF05, ENF06, ENF07	Noyau Linux, API interne, Stockage	T15, T16, T17, T19

Table 13 : Matrice de traçabilité

6. Analyse des défaillances et validation

6.1 Analyse des modes de défaillance (AMDEC simplifiée)

Situation de défaillance	Cause possible	Conséquence	Mesure préventive / corrective
Fausse alerte IDS	Configuration trop stricte (faux positif)	L'utilisateur perd confiance et désactive l'IDS.	Implémenter un système de "feedback" utilisateur pour ajuster la sensibilité (Machine Learning).
Saturation CPU/RAM	L'IA locale utilise trop	Le système d'exploitation	Implémenter un *cgroups* (Control Groups Linux) pour limiter les

Situation de défaillance	Cause possible	Conséquence	Mesure préventive / corrective
(Crash)	de ressources.	devient lent (freeze).	ressources allouées au processus de l'IA.
Perte de données AURAI	Coupure de courant lors de l'écriture en base.	Perte des tâches et objectifs.	Utiliser le mode WAL (Write-Ahead Logging) de SQLite et des sauvegardes automatiques.
Corruption du noyau et systemd	Mise à jour système défectueuse.	Système inutilisable (Kernel Panic).	Utiliser un système de fichiers à snapshot (ex: Btrfs) pour permettre un retour arrière (rollback).

Table 14 : AMDEC

6.2 Scénarios de test

- **T1 :** Lancer une attaque simulée (ex: arp-spoof avec Evillimiter) et vérifier que l'IDS génère une alerte en moins de 2 secondes.
- **T2 :** Déconnecter le câble réseau et vérifier que l'IA locale peut toujours résumer un document.
- **T3 :** Faire100 parties de jeu dans AURAI et vérifier que le IA évolue bien et ne présente pas d'incohérance données concernant les scores.
- **T4 :** Suivre un module CYBERDRIVE et vérifier que les scores sont bien persistés après redémarrage.

6.3 Stratégie de validation

La validation se fera via une approche **Design Science Research** en 3 phases :

1. **Validation interne :** Tests unitaires sur chaque module.
2. **Validation d'intégration :** Tests de bout en bout (scénario d'attaque -> formation -> planification).
3. **Validation externe :** Tests utilisateurs (panel de 5 à 10 personnes) avec questionnaires SUS (System Usability Scale) pour mesurer l'utilisabilité. Lancement produits Beta de GORAD OS, Marketing et ventes si Test réussi et produits fonctionelle et stable.

6.4 Défaillances complémentaires

Les quatre lignes du tableau 6.1 correspondent à R1 (fausse alerte), R2 (saturation CPU/RAM), R3 (perte de données AURAI) et R4 (corruption du noyau). Gravité (G) et probabilité (P) sont notées de 1 (faible) à 3 (élevée).

ID	Situation	Causes possibles	Conséquences	G / P	Mesures préventives ou correctives
R5	Attaque non détectée (faux négatif)	Règles limitées, surface d'observation incomplète	Fausse impression de sécurité	3 / 2	Surveillance multi-signaux (processus, ports, connexions, journaux), mise à jour des règles, limites de l'IDS indiquées à l'utilisateur
R6	Arrêt du service IDS	Plantage, manque de ressources	Perte de surveillance	3 / 2	Redémarrage automatique (systemd), alerte de surveillance dégradée, système anti-crash
R7	Réponse incorrecte	Limites du	Mauvaise décision de	3 / 2	Citation des sources, mention généré par IA,

ID	Situation	Causes possibles	Conséquences	G / P	Mesures préventives ou correctives
	ou inventée de l'IA	modèle	l'utilisateur		validation humaine
R8	Accès de l'IA à des données non autorisées	Permissions trop larges, erreur de configuration, Intrusion	Atteinte à la vie privée	3 / 2	Autorisation par source, journal des accès, Auto Destruction
R9	Injection de prompt via le chat	Entrée malveillant, JailBreak	Exécution de commandes non prévues	3 / 1	Isolation (sandbox) de l'IA, aucune exécution de commande par le chat
R10	Recherche documentaire locale incomplète	Indexation partielle (constaté au prototype : test de recherche locale partiellement réussi)	Résultats sans source pertinente	2 / 2	Réindexation, test de rappel sur requêtes types
R11	Missions CYBERDRIVE incomplètes	Conception narrative longue	Perte de la dimension aventure	2 / 3	Priorité de développement, au moins une mission par niveau
R12	AURAI perçu comme	Collecte excessive, rappels	Rejet de l'outil	2 / 2	Consentement, fonctions désactivables, collecte minimale

ID	Situation	Causes possibles	Conséquences	G / P	Mesures préventives ou correctives
	intrusif	insistants			

Table 15 : Défaillances complémentaire

6.5 Méthodes de vérification

- Tests fonctionnels de chaque exigence (extension des tests déjà réalisés).
- Tests de performance : CPU, mémoire et temps de réponse sur le matériel cible pour obtenir configuration nécessaire pour tourner GORAD OS.
- Simulation d'événements pour mesurer la détection, les faux positifs et le temps de détection de l'IDS.
- Tests hors ligne et capture réseau (Wireshark, IDS-GORAD) pour ENF01 et ENF02.
- Revue de configuration de sécurité (privilèges, isolation, permissions).
- Questionnaire SUS et observation d'utilisateurs externes, retour de GORAD community.
- Revue de conception pour la maintenabilité et Mis à jours.

6.6 Scénarios de test complémentaires

Les tests T1 à T4 sont décrits en 6.2.

ID	Scénario	Résultat attendu	Exigence
T5	Lancer un processus inconnu (exemple : Goradybe.exe ou	Alerte générée	EF01, EF02

ID	Scénario	Résultat attendu	Exigence
	Goradybe.elf)		
T6	Ouvrir un port non autorisé par netcat (nc -lnvp 9999 -e /bin/bash)	Alerte affichée en 2 secondes maximum	EF01, EF02
T7	Demander l'explication d'une alerte	Explication claire avec recommandation	EF05
T8	Cliquer sur une alerte, accepter puis refuser la leçon et la tâche proposées	Leçon ouverte et tâche créée seulement après confirmation	EF06, EF09
T9	Valider un quiz CYBRDRIVE et comparer avec le suivi AURAI	Score enregistré, valeurs identiques	EF03, EF04
T10	Créer, modifier, supprimer une tâche et un objectif	Données correctes et persistantes	EF04
T11	Désactiver une source puis interroger l'IA	Source jamais lue, accès journalisés	EF08
T12	Capturer le réseau pendant 30 minutes d'usage de l'IA	Aucune connexion sortante	ENF02
T13	Accès d'un utilisateur non autorisé à la configuration de l'IDS (Piratage)	Accès refusé	EF07
T14	Vérifier les privilèges de chaque service	Aucun service applicatif en root.	ENF03
T15	Mesurer CPU et RAM au repos pendant 1 heure	Au plus 10 % de CPU et 1 Go de RAM (hors IA), Traffic réseau stable	ENF07
T16	Arrêter de force le service IDS	Redémarrage en 30	ENF06

ID	Scénario	Résultat attendu	Exigence
		secondes maximum	
T17	Couper brutalement l'alimentation pendant une écriture	Aucune perte de données validées	ENF06
T18	Questionnaire SUS auprès de 5 à 10 utilisateurs	Avis positif et recommandation pris en compte	ENF04, ENF08
T19	Remplacer un module de test	Autres modules inchangés et fonctionnels	ENF05
T20	Arrêter le modèle de langage pendant une requête	Message de repli, autres modules actifs	R7, section 4.5
T21	Soumettre un événement mal formé à l'IDS	Rejeté et signalé sans plantage	Section 4.5
T22	Générer 100 alertes en 10 secondes	Interface réactive, alertes regroupées	Section 4.5

Table 16 : Scénarios de test complémentaires

6.7 Critère de validation globale

Le système est considéré conforme aux besoins initiaux lorsque : toutes les exigences de priorité Critique sont vérifiées ; aucun risque de gravité 3 n'est sans mesure en place ; le score SUS atteint minimal requis; l'expérimentation de l'IDS (taux de détection, taux de faux positifs, temps de détection) a été menée sur un volume d'événements suffisant, ce qui n'est pas le cas au prototype actuel.

7. Proposition d'architecture générale

7.1 Vue architecturale (en couches)

L'architecture de GORAD OS est basée sur un modèle en couches :

1. **Couche Matérielle & Noyau :** Noyau Linux standard stable.
2. **Couche Services Système :** Démon IDS (surveillance), Démon IA (serveur d'inférence local), Gestionnaire de base de données (SQLite).
3. **Couche API Interne (Middleware):** Un bus de communication permettant aux applications de discuter entre elles de manière sécurisée.
4. **Couche Application (UI) :** Interface utilisateur unifiée (ex: Electron, Qt ou interface web locale) intégrant les modules IDS GORAD, CYBERDRIVE et AURAI. Themes style GORAD intégré.

7.2 Technologies envisagées et justifications

Technologie	Rôle	Justification
Linux (Debian)	Base OS	Stabilité, support matériel étendu, documentation abondante sur le noyau.
Python (django)	Backend / API	Excellent pour l'IA, rapide à développer, gestion des API native.
SQLite	Base de données	Zéro configuration, local, parfait pour une application de bureau.
Ollama / Llama.cpp	Moteur IA	Permet d'exécuter des LLM (ex: Mistral, Llama 3, Qween 1b) en local sans connexion.
QT, javascript , css	Interface Utilisateur	Permet une interface moderne et réactive, tout en restant multiplateforme (packagé en desktop pour certains application tierces [ex: GoradFiles:

Technologie	Rôle	Justification
		Gestionnaire de fichiers orientés cyber).

Table 17 : Architecture de GORAD OS

7.3 Mesures de sécurité, fiabilité et maintenabilité

- **Sécurité :**
  - Isolation des processus : Le module IA tourne dans un conteneur (Docker) ou un *sandbox* pour ne pas accéder à tous les fichiers de l'utilisateur.
  - Principe du moindre privilège : IDS GORAD tourne avec un utilisateur système dédié, pas en root sauf pour les processus nécessitant sudo.
- **Fiabilité :**
  - Journalisation centralisée (rsyslog ou journald) pour tracer les erreurs.
  - Mécanisme de "Watchdog" pour redémarrer les services défaillants (ex: systemd).
- **Maintenabilité :**
  - Code modulaire respectant les principes SOLID[3].
  - Documentation technique et Conception bien écrit.

[3] SOLID est un acronyme regroupant cinq principes de conception en programmation orientée objet, popularisés par Robert C. Martin ("Uncle Bob"). Ils visent à rendre le code maintenable, extensible et testable en réduisant les dépendances rigides entre les classes.

7.4 Composants logiciels et responsabilités

Composant	Responsabilité	Interactions principales
Collecteurs système (consumers.py) en mode async	Lire journaux, processus, ports et connexions	Alimentent l'IDS (F1)
Moteur IDS GORAD	Comparer aux règles, qualifier, générer les alertes	Écrit dans le stockage, notifie l'API (F2, F3)
Serveur applicatif et API interne	Authentification, routage, validation, temps réel vers l'interface	Relie interface, modules et stockage
Module CYBERDRIVE	Contenus pédagogiques, scores, progression	API, stockage (F6, F7)
Module AURAI	Tâches, objectifs, bien-être, suivi	API, stockage, lecture de la progression de CYBERDRIVE (F7, F8), LLMA CPP
Moteur d'IA locale	Exécution du modèle de langage et recherche documentaire	API uniquement, sources autorisées (F4, F5, F9)
Gestionnaire de permissions et d'audit	Rôles, consentements, journal des accès	Consulté par tout accès aux données
Interface locale (tableau de bord)	Affichage et confirmations	API

Table 18 : Composants logiciels

7.5 Interactions entre composants

Les flux montent des collecteurs système vers l'interface et les commandes descendent par l'API interne. Les modules ne s'appellent pas directement entre eux : toutes les communications (F1 à F9 de la section 4.4) passent par l'API, qui vérifie les autorisations auprès du gestionnaire de permissions. Cette règle limite l'impact d'une panne : si CYBERDRIVE ou l'IA s'arrête, l'IDS continue de protéger la machine.

7.6 Organisation des données et flux principaux

Jeu de données	Contenu	Producteur	Consommateurs
Événements et alertes	Date, type, machine, processus, adresse, priorité, état	IDS GORAD	Tableau de bord, IA locale, AURAI
Comptes et rôles	Identifiants, rôles, préférences de confidentialité	Gestionnaire de permissions	Tous les modules
Contenus pédagogiques	Modules, leçons, exercices, missions, quiz	CyberGorad	CYBERDRIVE, IA locale (recherche)
Progression	Scores, niveaux, missions validées, Aures gagné dans AURAI	CYBERDRIVE	AURAI, tableau de bord
Données d'organisation	Objectifs, tâches, notes, calendrier, résultats des exercices	AURAI	Utilisateur, IA locale si autorisée
Journal	Accès de l'IA, actions	Gestionnaire de	Utilisateur,

Jeu de données	Contenu	Producteur	Consommateurs
d'audit	automatiques	permissions	administrateur
Index documentaire local	Représentation des documents pour la recherche	IA locale	IA locale

Table 19 : Organisation des données et Flux

8. État d'avancement du prototype et perspectives

Le prototype confirme la faisabilité technique : 8 des 13 fonctionnalités recensées sont intégrées, 4 sont partiellement intégrées (surveillance des connexions réseau, missions CYBERDRIVE, suivi de progression AURAI, recherche dans les documents locaux) et 1 est en développement (explication des alertes par l'IA). 8 tests fonctionnels sur 10 sont réussis ; les 2 tests partiellement réussis concernent le fonctionnement local (recherche documentaire et utilisation hors ligne). CYBERDRIVE contient 8 modules, 54 leçons, 20 exercices et 30 quiz, mais les missions restent à compléter. IDS GORAD a traité 1 processus critique, 1 connexion réseau et 4 ports ouverts, et généré 3 alertes de ports non autorisés et 2 de processus inconnus.

Limites : prototype à un stade précoce ; volume d'événements trop faible pour calculer des taux fiables ; AURAI ne possède que 2 jeu; aucune évaluation par des utilisateurs externes avant lancement produits ; aucune mesure quantifiée de l'IA locale ; robustesse sur la durée non testée, Limites matériels , le PC de cybergorad n'est que core i-5 3 ème 8GB RAM mais au moins possèdant disque SSD.

Priorités pour la suite : (1) finaliser les composants partiels et réaliser les exigences non encore couvertes (explication des alertes, lien alerte vers

leçon, tâche corrective sur confirmation, contrôle des sources de données) ; (2) expérimenter l'IDS à plus grande échelle ; (3) tester l'IA locale et évaluer l'ensemble avec des utilisateurs, Test de performances, améliorer les cours et la pédagogie utilisé des cours, écriture de l’histoire de cybergorad dans CYBERDRIVE.

Conclusion

Ce document prouve que GORAD OS n'est pas une utopie. Un environnement unifié, sécurisé et souverain est techniquement réalisable et il répond à un besoin réel, concret et actuel de l'utilisateur moderne.

Références

- *Documentation de sécurité du noyau Linux (kernel.org), notamment seccomp et Landlock.*
- *Article de recherche GORAD OS (Rakotoarivony A. T. A.), source des informations sur le prototype.*